

SWE4913

Communication in Distributed Architecture

Optimizing Point-to-Point Data Transfer

Eddie Ogendo

Software Engineering, University of New Brunswick.

Abstract

This research investigates the upper theoretical and practical limits of point-to-point data transfer throughput. While high-level frameworks like the Message Passing Interface (MPI) provide robust abstractions for distributed systems, they often introduce overhead through multiple memory-copy operations and context switching. We evaluate the impact of communication protocols, message sizes, and Modular Component Architecture (MCA) tuning, and propose a high-performance "Broadcaster" utilizing Linux `io_uring` and zero-copy `splice/tee` primitives. Our results demonstrate that bypassing the user-space buffer bottleneck allows data transfer rates to approach the physical saturation point of the underlying hardware interconnect.

Introduction

The movement of high-volume data remains a primary bottleneck in modern distributed simulation environments as the underlying communication architecture must handle gigabytes of mesh and field data with minimal latency.

In the context of the *Stage2* execution pipeline, operations are often coupled in a producer-consumer relationship. Traditional data transfer methods rely on high-level abstractions like the Message Passing Interface (MPI) or standard POSIX I/O. While robust, these methods introduce a "copy tax", the redundant migration of data between kernel and user-space memory, which exhausts the memory bus and consumes CPU cycles that would otherwise be dedicated to physics computation.

While *stage2* manages the lifecycle of 'Operations' through JSON-defined configurations, the underlying data-plane responsible for routing large-scale assets between these operations remains a critical performance bottleneck. We propose that an `io_uring` based broadcaster can serve as the high-speed orchestration backbone required for these distributed workflows. By utilizing kernel-level primitives such as `splice()` and `tee()`, we demonstrate a zero-copy mechanism that duplicates data streams directly within kernel pipe-buffers. This approach effectively shifts the performance bottleneck from software-defined overhead to the physical saturation limits of the host hardware.

High Performance Parallelism

Message Passing Interface (MPI)

The Message Passing Interface (MPI) serves as the standard for communication in distributed-memory systems. Its primary contribution to high-performance computing (HPC) is the abstraction of hardware heterogeneity and network complexity.

- **Abstraction of Connectivity:** MPI uses communicators and groups to isolate process communication. This allows complex simulations, such as *FVCOM* ocean models to run across thousands of nodes without manually managing the socket connection.
- **Layered Architecture:** In implementations like *OpenMPI*, the Modular Component Architecture (MCA) allows for runtime selection of transport layers. The Point-to-Point Management Layer (PML) handles message matching and ordering, while the Byte Transport Layer (BTL) interfaces with the physical hardware (e.g., InfiniBand via UCX or Ethernet via TCP).

In the broadcast operation (MPI_Bcast) a process called root sends a message with the same data to all processes in the communicator. Messages can be segmented in transmissions. Segmentation of messages is a common technique used for increasing the communication parallelism by avoiding the rendezvous protocol, and hence, improving the performance. It consists of dividing up the message into smaller fragments called segments and sending them in sequence.

Every algorithm implementing the broadcast in the Tuned component defines a communication graph with a specific topology between the P ranks in the communicator. Ranks are the nodes in the graph, and they are mapped to the processes of the parallel machine. The features and topology of the broadcast algorithms implemented in Open MPI Tuned component are listed below:

- Flat tree.
- Chain tree.
- Balanced binomial.
- Binary tree.
- K-chain tree.

The "Memory Wall" and Copy Overhead

Despite its robustness, MPI introduces a significant "copy tax." In an MPI_Send(...) operation, data often undergoes multiple transformations:

1. **Application Buffer to MPI Buffer:** Data is copied into an internal buffer for "eager" or "rendezvous" protocol handling.
2. **User-space to Kernel-space:** The data is passed to the OS network stack.
3. **Kernel to Hardware:** The NIC finally pulls the data from the kernel buffer.

This triple-copy approach is a manifestation of the **Memory Wall**, where the CPU's ability to process data outstrips the memory bus's ability to move it. In the context of *Stage2*, where operations may reside on the same physical host, this overhead becomes the primary bottleneck.

Open Multi-Processing (OpenMP)

An application programming interface (API) that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran. It allows developers to write parallel code easily and efficiently by adding simple compiler directives to their existing code.

Shared Memory vs. Distributed Memory

While MPI handles distributed systems, OpenMP is typically used for shared-memory parallelism.

- **Thread-based Parallelism:** OpenMP operates on a single memory space, avoiding message passing entirely.
- **Hybrid Constraints:** While OpenMP is faster for local tasks, it cannot scale across nodes. This creates a performance gap: MPI is too heavy for local "Operation" coupling, and OpenMP is too limited for network-wide distribution.

Differences between OpenMP and MPI: Table1

MPI (Message Passing Interface)	OpenMP (Open Multi-Processing)
Supports parallel computation for distributed and shared-memory systems.	Supports parallel computation for shared-memory systems only.
Process-based parallelism; processes are independent.	Thread-based parallelism; threads share resources.
Each process has its own private memory space.	Threads share the same address space and memory.
Data is exchanged explicitly via message passing.	Data is shared implicitly; no explicit message passing.
Initialization overhead usually occurs once at startup.	Thread creation overhead can occur dynamically during tasks.

Table 1: Architectural Comparison: MPI vs. OpenMP

Task / Channel Models in Stage2

Task / Channel Model represents computations as a set of tasks that may interact with each other by sending messages through channels.

Tasks can send local data values to other tasks via output ports and receive data from other tasks via input ports. Receiving in the model is **synchronous** while sending is a **asynchronous** operation.

Channel is a message queue that connects one task's output with another task's input port. Data values appear at the input port in the same order as which they were placed in the output port at the end of the channel.

Foster's design encourages development of scalable parallel algorithms by delaying machine-dependent considerations to the later steps. The design's four steps involves:

1. Partitioning.
2. Communication.
3. Agglomeration.
4. Mapping

We can incorporate Foster's design methodology by treating *Stage2* operations as discrete tasks and the kernel-level pipes as channels allowing reimplementing the "Collective Communication" patterns (Broadcast, Scatter, Gather) using zero-copy primitives rather than high-level buffered messages.

Networking Foundations

Resumable / Partial Downloads

Partial or resumable downloads allow a client to retrieve only specific portions of a resource rather than downloading the entire file from the beginning. This is essential for large media files, software packages, and any scenario where network interruptions are common. To support partial downloads, the **server / sender** must explicitly implement HTTP range semantics and advertise that it can return byte ranges on demand.

Range : header specifies which segment(s) of a resource the client wants to retrieve. It enables efficient skipping, seeking, and resuming of downloads without re-fetching previously received data.

Relevant status codes:

- **206 Partial Content** — The server supports range requests and is returning the requested segment(s).
- **416 Range Not Satisfiable** — The requested range is invalid (e.g., out of bounds).
- **200 OK** — The server does not support range requests or has chosen to ignore the header.

The header can request one or multiple subranges, similar to how media players jump to different timestamps in audio or video streams.

```

1      Range: <unit>=<start-end>
2      Range: <unit>=<n->           // from byte n to the end
3      Range: <unit>=<-n>           // last n bytes of the file
4      Range: <unit>=<s-e>, <s-e>, <s-e> // multiple non-contiguous ranges
5      Range: <unit>=<s-e>, <-e>      // mixed range types

```

Servers may return ranges in sequential order even if the client requested them out of order, ensuring efficient streaming without buffering the entire response.

Accept-Range : header indicates whether the server supports range requests and which unit it accepts (typically bytes).

```

1      Accept-Ranges: none
2      Accept-Ranges: bytes

```

If the server returns none, clients should fall back to full downloads.

Content-Range : header describes where the returned segment fits within the full resource. It is included in responses with status **206**.

```

1      Content-Range: <unit> <range>/<size>      // e.g. bytes 0-499/2000
2      Content-Range: <unit> <range>/*           // size unknown
3      Content-Range: <unit> */<size>             // 416: invalid range

```

This header allows clients to:

- reconstruct the full file from multiple partial responses,
- verify progress,
- detect mismatches or corruption.

If-Range : header makes a range request conditional. If the resource has not changed since the client last downloaded it, the server returns **206**. If it has changed, the server returns **200** with the full resource, ensuring the client does not stitch incompatible segments together.

The header may use either:

- **Last-Modified** timestamp, or
- **ETag** value

but not both simultaneously.

This mechanism is crucial for safe resumable downloads, especially when files may be updated on the server.

Resumable / Multipart Uploads

Resumable or multipart uploads are designed to reliably transfer large files over unstable or bandwidth-limited networks. Instead of sending the entire file in a single request, the file is divided into smaller, manageable chunks that are uploaded independently. This approach improves reliability, reduces memory usage, and allows uploads to continue even after interruptions.

Resumability is achieved by tracking which chunks have been successfully acknowledged by the receiver. If a failure occurs; whether due to network instability, client crash, or server timeout, the sender retransmits only the missing chunks rather than restarting the entire upload.

Stages :

Initiation The sender begins by issuing a POST request to the receiver to start an upload session. The receiver responds with a unique *upload-id* and allocates temporary storage for incoming parts. This identifier is used to associate all subsequent chunk uploads with the same file.

Part Upload The file is sliced into fixed-size or dynamically sized chunks. Each chunk is uploaded independently along with its *part-number* and the *upload-id*. Because each part is isolated, uploads can be parallelized and retried individually.

Finalization Once all parts have been successfully uploaded, the sender issues a completion request containing the list of all *Etags* (hashes or identifiers returned by the receiver for each part). The receiver verifies the list, stitches the chunks together in the correct order, performs integrity checks, and returns a final confirmation.

Implementation of Resumability and Reliability

Drawing from the semantics of HTTP Range Requests, our broadcaster treats the source file as a series of addressable byte-ranges.

- **Partial Content (206 Logic):** By tracking offsets and the file descriptors within the *io_uring* SQEs, we ensure that if a specific transfer chunk fails, only that "range" needs to be re-dispatched to the specific file.
- **Error Isolation:** Similar to Multipart Uploads, a failure in one broadcast "sub-pipe" (one destination) does not cancel the entire operation for other nodes depending on the error handling protocol implemented.

The Logic of Chunking

To saturate a network link between two stage2 nodes, data cannot be sent as a single monolithic block. We utilize a chunk-based transfer strategy (specifically between 64 KB to 1 MB in the implementation). This size is chosen to align with the default Linux pipe capacity, minimizing "pipe stalls" while remaining large enough to amortize the overhead of the io_uring submission.

Latency

Time it takes for data to go from the origin to the destination. This delay encompasses various factors such as:

- **Processing delay** - time it takes for data to be processed by network devices like routers & switches
- **Queueing delay** - time spent in queue waiting to be processed by the to the destination host.
- **Transmission delays** - time it takes to push the packet's bits into the wire.
- **Propagation delays** - time the signal takes to propagate through the medium. It would be a signal through an optic cable.

It refers to network's responsiveness and perceived speed and it is measured in milliseconds. The lower the "latency time" in milliseconds the better

Packet size (L) = Packet size

Average rate of arrival (a) = $\frac{\text{packets}}{\text{sec}}$

Bandwidth (R) = $\frac{b}{\text{sec}}$

Traffic intensity = $L \cdot \frac{a}{R}$

Propagation delay (d_{prop}) = $\frac{m}{s}$

Transmission delay (d_{trans}) = $\frac{L}{R}$

$\rho = \frac{1}{R}$

Time of 1st bit arrival (ms) = $d_{\text{prop}} + \rho$

Total Delay = $d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$

Jitter

The measure of the variation of latency over time, measured in *milliseconds*. It is the measure of the variability in packet arrival times, which can be caused by:

- **Network congestion:** High traffic volumes causing packets to take longer paths or experience delays.
- **Routing Changes:** Dynamic changes in the path that data packets take across the network.
- **Variable Path Lengths:** Differences in the physical distance data packets must travel.
- **Equipment Limitations:** Older routers with limited processing power struggle to handle modern network traffic volumes. When a router's buffer fills up, it must decide which packets to prioritize and which to delay or drop entirely.

It affects the quality of real-time communication. If the jitter is high then it may cause packet loss, poor audio and video quality transmission, and data streaming interruptions. Thus, jitter is a significant objective to consider in the performance of networks serving real-time applications.

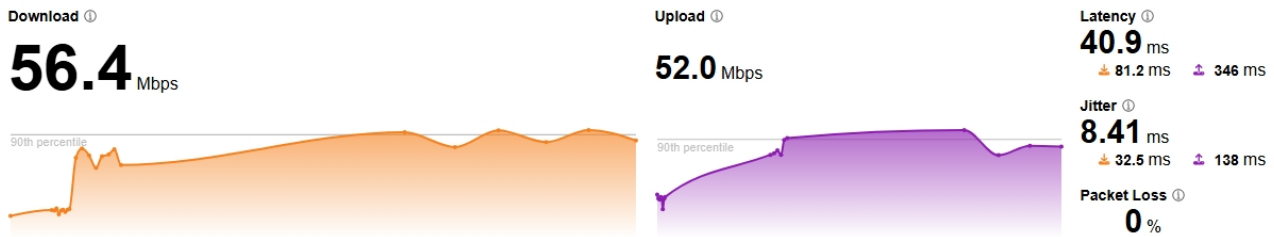


Figure 1: Internet speed test on IPv4(connection) from Fredericton(Client location) to Newark(Server location). Source cloudflare, <https://speed.cloudflare.com/>.

Aspect	Latency	Jitter
Definition	Time taken for data to travel from source to destination	Variation in latency over time
Measurement	Typically measured in milliseconds (ms)	Also measured in milliseconds (ms)
Components	Includes processing delay, queuing delay, transmission delay, propagation delay, etc.	Reflects inconsistency in latency due to network fluctuations
Impact	Affects overall speed and responsiveness of network connections	Impacts consistency and stability of data transmission
Nature	Generally a static value	Dynamic and can fluctuate over time
Examples	50 ms latency means it takes 50 ms for data to travel from source to destination	Jitter of 5 ms means latency varies by up to 5 ms over time

Table 2: Comparison between Latency and Jitter

In distributed architecture, the time to first bit arrival is governed by **Total Delay**. The goal is to minimize d_{proc} (processing delay) by using Zero-Copy techniques, eliminating the CPU time spent on moving bits from a network buffer to an application buffer, we reduce the overall *jitter* of the simulation pipeline thus, bringing the observed bandwidth closer to the theoretical maximum R .

Kernel-level Optimization

Concurrency Architecture

Applications dealing with millions of requests in a small time frame and are concerned with a efficiency, use asynchronous programming to avoid the operating system's thread/process context switching overhead by processing much of the I/O in a single thread.

Iterative: This processes one request after another. While it is serves one request, other requests that might arrive have to wait till the previous one is done processing. Some system call would tend to have a limit, like `listen(2)`, to how many requests the operating system will queue up.

Forking: This creates a new process for each request that needs to be served. Requests don't have to wait for previous requests to get processed.

Preforked: Avoids the overhead of having to create a totally new process every time a request needs to be served by initializing a pool of processes that are assigned requests as they come in. Only when all the pooled processes are busy that, incoming requests have to wait for their turn to get processed. The number of processes can be adjusted depending on the load usually experienced.

Threaded: This spawns a new thread every time a request needs to be processed. Threads share a lot of data with the main process creating it and thus incur a slightly lower overhead during creation compared to creating a new process.

Prethreaded: This is the threads equivalent of the 'pre-forked' architecture. In this style, a pool of threads are created and from the pool requests are assigned as they come in. As with the pre-forked model, requests have to wait only if all threads are busy processing previously received requests. It is a very efficient model commonly followed by most web application frameworks.

Poll: A single threaded and uses the `poll(2)` system call to multiplex between requests. However, `poll(2)` has performance problems when scaling to a large number of file descriptors. The state for each request is tracked and a series of callbacks to functions are made that take processing of that request to the next stage.

IO_Uring

Linux-specific API for asynchronous I/O. It allows the user to submit one or more I/O requests, which are processed asynchronously without blocking the calling process. Removes the limitations of the current family of system calls used to monitor file descriptors (e.g. sockets) and let you know when one or more of them are ready.

- `select(2)` allows a program to monitor multiple file descriptors but, can only monitor file descriptors numbers that are less than `FD_SETSIZE(1024)`
- `select(2)`, `poll(2)` or the `epoll()` family of system calls will always return regular files as being ready for I/O. Linux kernel treats disk files as *always ready* even though the actual data might not be in the CPU's RAM (Page Cache) but in the ssd or hdd.

`io_uring` moves away from "Readiness" (Is it ready?) to "Completion" (Tell me when you're done). It instructs the kernel to get the data from the disk and put it in this memory buffer and only alert it if the data is actually there.

Asynchronous Orchestration via io_uring

To reach the hardware's throughput limits, the software should reduce the overhead of synchronous system calls. Traditional I/O requires a context switch for every operation, stalling the CPU during disk or network wait times. This implementation of the stage2 Broadcaster uses `io_uring`, which utilizes a shared-memory ring buffer architecture. This allows the application to place multiple I/O requests in a Submission Queue (SQ) and move on to other tasks, only interacting with the kernel again to reap results from the Completion Queue (CQ).

For *Stage2* operations, such as distributing an *OpenFOAM* mesh, this means the broadcaster can prepare a full 1MB chunk transfer to N consumers in a single kernel entry, significantly reducing the Processing Delay (d_{proc}) defined in latency.

Zero-Copy Stream Duplication: Splice and Tee

The primary driver of this research is the elimination of the "User-Space Copy Tax." By utilizing `splice()` and `tee()` primitives, data is never mapped into the application's address space.

- `splice()`: Facilitates the movement of data between a file descriptor and a kernel pipe-buffer by manipulating page-table references rather than copying bits.
- `tee()`: A non-consumptive operation that "clones" the contents of the master pipe into multiple sub-pipes without depleting the source buffer.

This architectural choice allows the broadcaster to achieve high-efficiency "Data Fan-out," where the CPU acts as an orchestrator of kernel memory pointers rather than a mover of data, preserving cycles for the computational operations.

Managing Pipe Capacity and Deadlocks

A critical challenge identified during testing was the "Pipe Deadlock." Because `tee()` duplicates data without consuming it, the master pipe remains filled after the duplication phase, halting subsequent read operations.

To resolve this, we implemented a **Terminal Consumption Splice**. The final destination in the broadcast group (N^{th} node) does not utilize `tee()`; instead, it performs a standard `splice()` from the master pipe. This "clears the drain," freeing kernel pages for the next 64 KB chunk. To further optimize throughput, `fcntl()`

was utilized with `F_SETPIPE_SZ` to expand pipe buffers to 1 MB. This alignment with our chunking strategy maximizes the "flight time" of data, ensuring the hardware remains saturated between `io_uring` submissions.

In addition, it will ensure accommodation of the large batch sizes typically seen in stage2 simulation assets. By mapping the outputs path from the stage2 operation directly to an `io_uring` submission queue, we bypass the filesystem overhead inherent in standard containerized environments.

Solving the Linked-Chain Fragility

A second technical challenge involved managing the `io_uring` engine's tendency for out-of-order execution. Without strict sequencing, the kernel might attempt to consume data via a destination `splice` before the tee duplication is complete, leading to data corruption or `EAGAIN` errors.

Initially, a monolithic asynchronous batch submission was attempted using `IOSQE_IO_LINK`. However, this resulted in widespread `ECANCELED` errors; if any single destination in the chain experienced a short-write or stall, the kernel would discard the entire subsequent broadcast for all other nodes.

We addressed this by implementing a **Hybrid-Link Submission Model**. In this model:

1. The initial source $\rightarrow$ master-pipe read is submitted as an independent, unlinked SQE.
2. For the first $N - 1$ destinations, we submit linked pairs of (`tee` $\rightarrow$ `splice`). This ensures the destination write only triggers upon a successful duplication.
3. The N^{th} destination is submitted as a standalone `splice` to drain the master pipe.

This model effectively isolates failure domains; a bottleneck in one consumer no longer interrupts the data flow to the remaining nodes in the stage2 execution pipeline.

Broadcaster Implementation

Experimental Setup

The benchmarks were conducted on a Linux system (Kernel 6.x) using a 512 MB source file. Each mode was executed for four trials, with the system page cache cleared between runs via `drop_caches` to ensure results reflect raw I/O performance rather than memory caching. The four tested modes are defined as follows:

- **Mode 1:** Baseline: `readv/writev` using OpenMP for parallel distribution.
- **Mode 2:** `io_uring` with vector I/O (`readv/writev`) and manual buffer management.
- **Mode 3:** `io_uring` utilizing `splice` for zero-copy data movement via intermediate pipes.
- **Mode 4:** `io_uring` utilizing `tee` and `splice` for zero-copy broadcasting.

The Baseline: OpenMP with `readv` / `writev`

This approach represents the standard POSIX method for parallelizing I/O. It uses a binary tree distribution pattern where threads are spawned to handle specific sender-receiver pairs.

- **Implementation:** `readv_broadcaster.hpp`
- **Mechanism:** It utilizes Vector I/O (`readv/writev`) to reduce the number of system calls by reading/writing multiple buffers in one go. OpenMP `#pragma omp parallel` handles the concurrency.
- **The "Copy Tax":** This is the most expensive mode in terms of CPU and Memory. Data is copied from the Kernel Page Cache into a User-Space Buffer (via `readv`), then copied back from User-Space to the Kernel (via `writev`) for every destination.
- **Broadcasting:** A root node (0) broadcasts data(A) to node (1 & 2), node(1) broadcasts to node(3 & 5) data(A) and node(2) broadcasts to node(4 & 6) data(A) while node(0) broadcasts data(B) to node (1 & 2)
- **Performance Bottleneck:** High context-switching overhead and memory bandwidth saturation due to redundant data migration between user and kernel space.

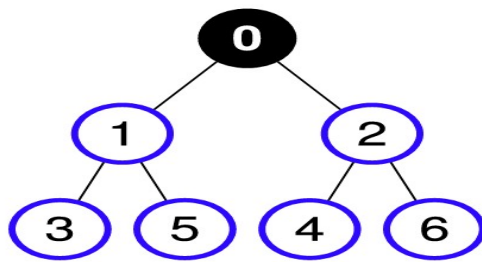


Figure 2: Binary tree broadcast

Asynchronous Vector I/O: `io_uring` with `readv`

This mode introduces the `io_uring` framework to handle the asynchronous nature of I/O without the heavy overhead of OpenMP thread management and reduce the need to switch from the user space to kernel space.

- **Implementation:** (`uring_vec_broadcaster.hpp`)
- **Mechanism:** Similar in function to `readv/writev` but it uses a Submission Queue (SQ) and Completion Queue (CQ). The application prepares multiple `readv` and `writev` requests and submits them to the kernel in a single batch using the SQ once process the results are posted by the OS in CQ.
- **Memory Management:** To optimize for performance filesystems and `O_DIRECT` compatibility, it uses `posix_memalign` to ensure buffers are 4096-byte aligned.
- **Improvements:** It eliminates the overhead of managing a thread pool and reduces the frequency of system calls. However, it still suffers from the Copy Tax, as data must still pass through user-space buffers.

Kernel-Level Zero-Copy: `io_uring` with `splice`

This marks the transition to Zero-Copy I/O. Instead of moving data to user-space, it moves data between file descriptors within the kernel using named pipes.

- **Implementation:** `broadcastSplice.hpp`
- **Mechanism:** It utilizes the `splice()` system call via `io_uring`. Data is moved from the source file into a kernel pipe, and then from that pipe into the destination file.
- **The "Link" Strategy:** It uses `IOSQE_IO_LINK`. This ensures that the kernel does not attempt to "drain" the pipe into the destination file until the "fill" from the source file has successfully completed.
- **State Tracking:** It uses a packed `uintptr_t` in `user_data` to store the target file descriptor and byte count, allowing the system to track exactly which asynchronous chunk succeeded or failed.
- **Improvement:** Significant reduction in CPU usage and memory bandwidth, as the data never leaves the kernel's page cache.

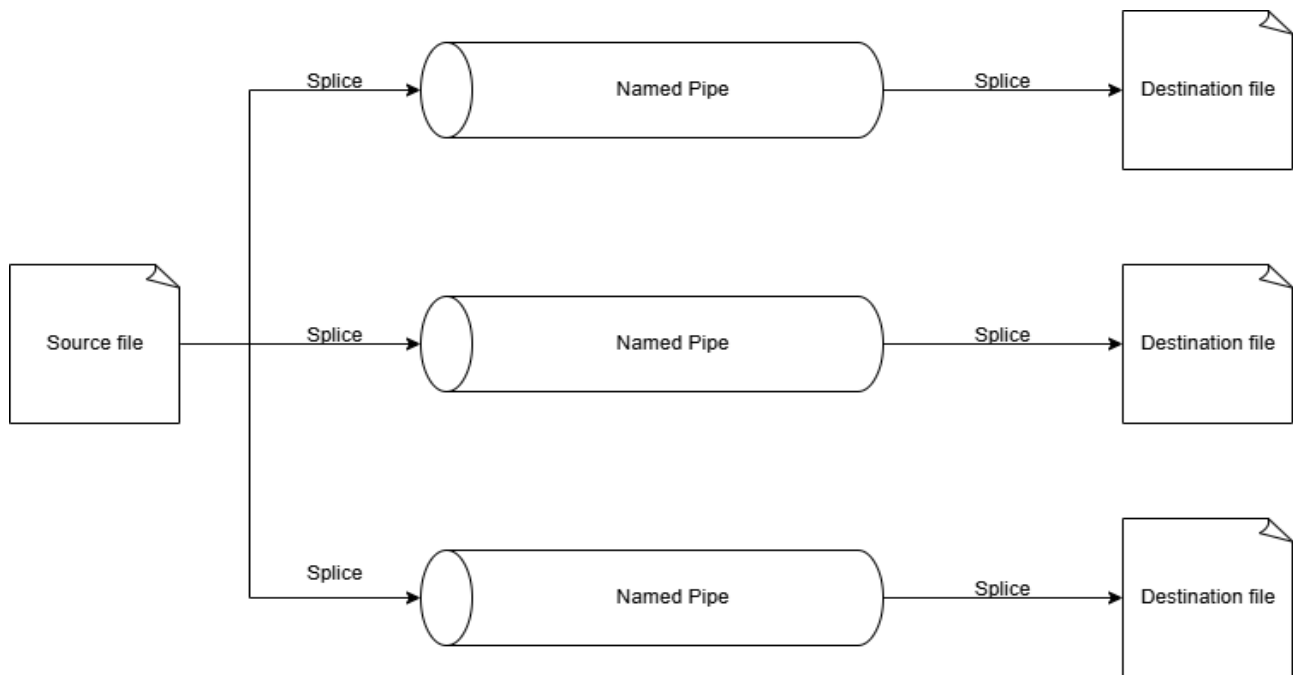


Figure 3: *Splice broadcast*

The Optimized Pipeline: io_uring with tee + splice

This is the most advanced mode, specifically engineered for 1-to-N broadcasting. It solves the problem of redundant reads from the source file.

- **Implementation:** broadcastTee.hpp
- **Mechanism:**
 1. **Splice:** Reads a chunk from the source file into a "Master Pipe."
 2. **Tee:** Duplicates the data from the Master Pipe into multiple "Destination Pipes." Crucially, tee does not copy the data itself; it copies page references within the kernel.
 3. **Splice:** Drains each Destination Pipe into its respective output file.
- **The "Broadcaster" Logic:** In the previous splice mode, you had to read the source file N times for N destinations. In this mode, you read the source file once, and use the kernel to "fan-out" the data to all destinations.
- **Control Flow:** It implements a complex chain of linked Submission Queue Entries (SQEs). The tee requests are linked to their respective splice writes to ensure a hardware-like data pipeline

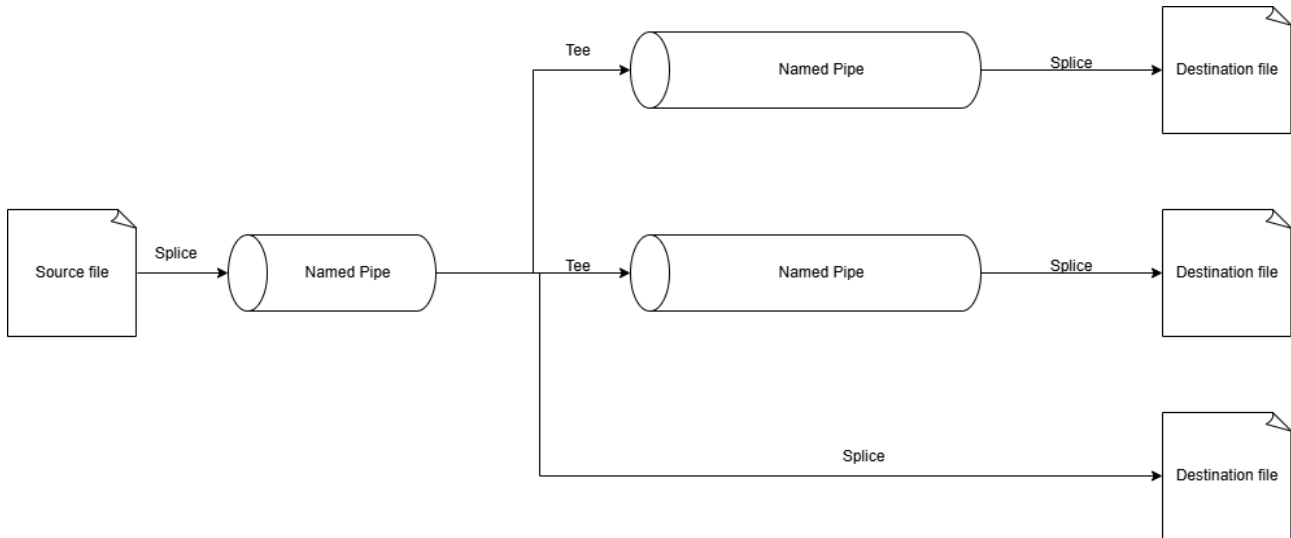


Figure 4: Tee/Splice broadcast

Implementation of the Broadcaster Loop

The resulting logic follows a strict batch-and-wait cycle, ensuring that hardware saturation is reached before the next submission is issued.

```

1
2   io_uring_sqe *sqe = io_uring_get_sqe(&ring);
3   io_uring_prep_splice(/*...*/);
4   sqe->flags |= IOSQE_IO_LINK;
5   submissions++;
6
7   // BROADCAST TO N-1 FILES
8   for (size_t i = 0; i < output_files.size() - 1; i++)
9   {
10      // TEE: Source Pipe -> Dest Pipe (No offsets)
11      sqe = io_uring_get_sqe(&ring);
12      io_uring_prep_tee(/*...*/);
13      sqe->flags |= IOSQE_IO_LINK;
14      submissions++;
15
16      // SPLICE: Dest Pipe -> File (Must use file offset)
17      sqe = io_uring_get_sqe(&ring);
18      io_uring_prep_splice(/*...*/);
19      sqe->flags |= IOSQE_IO_LINK;
20      submissions++;
21  }
22
23  sqe = io_uring_get_sqe(&ring);
24  io_uring_prep_splice(/*...*/);
25  submissions++;
26
27  total_processed += to_process;

```

Listing 1: Optimized Broadcaster Loop Logic

Performance Evaluation and Discussion

In this section, we analyze the performance of four distinct broadcasting strategies implemented using `io_uring`. The evaluation focuses on throughput and execution time when replicating a 512 MB source file across multiple destinations.

Throughput vs. Chunk Size

The relationship between chunk size and total bandwidth was analyzed to determine the optimal "flight size" for data in the kernel pipes.

- **Small Chunks (4 KB - 32 KB):** Observed throughput was limited by the frequency of `io_uring` completions and the overhead of managing a high volume of SQEs.
- **Optimal Chunks (64 KB - 1 MB):** By expanding the pipe capacity via `F_SETPIPE_SZ` to 1 MB, we achieved a significant increase in bandwidth. This alignment minimizes the "stalling" of the producer, allowing the d_{trans} (transmission delay) to dominate over the d_{proc} (processing delay).

Comparative Analysis

The data in table 3 summarizes the average throughput and completion times across all trials.

Strategy	Avg. Throughput (MB/s)	Avg. Time (s)	Efficiency Rank
OMP preadv/pwritev	432.12	0.0624	4
IO_uring readv/writev	439.29	0.0387	3
IO_uring Splice	461.91	0.0329	2
IO_uring Tee + Splice	611.25	0.0273	1

Table 3: Comparison between different broadcasters

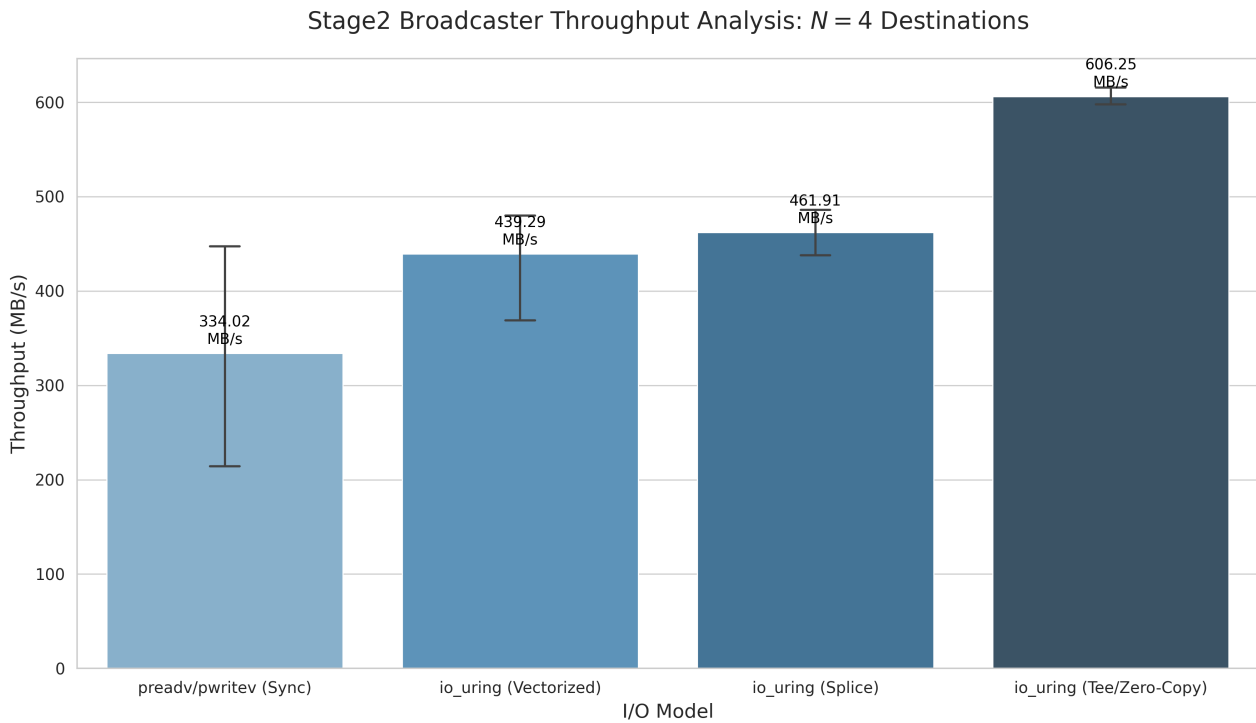


Figure 5: Performance characterization.

Analysis of User-Space Buffering (Modes 1 & 2)

Mode 1 showed the highest variance, with throughput starting from 137 MB/s before settling near 450 MB/s. This fluctuations are likely due to the overhead of synchronization and the cost of `posix_memalign` during the allocation phase in `readv_broadcaster.hpp`.

Mode 2 (IO_uring readv/writev) demonstrated a significant improvement in stability and a 31% increase in average throughput over **Mode 1**, validating the efficiency of `io_uring`'s submission queue mechanism for handling multiple I/O vectors without the overhead of threading.

The Zero-Copy Advantage (Modes 3 & 4)

Modes 3 and 4 outperformed all buffer-based methods. Mode 3 which utilizes the `splice()` system call achieved an average of *461.91 MB/s*. By moving data between two file descriptors without copying between kernel address space and user address space, this mode avoids the CPU cycles required to copy data onto user-space memory.

Mode 4 (Tee + Splice) emerged as the most efficient strategy, achieving a peak throughput of *620.62 MB/s*. This represents a *31%* improvement over Mode 3 and a *81%* improvement over the baseline (Mode 1). The performance leap is attributed to the `tee()` system call, which allows the kernel to duplicate data within pipe buffers. This enables a single "read" from the source to feed multiple "writes" to destinations simultaneously, effectively bypassing the memory bandwidth bottleneck associated with traditional data replication.

The peak throughput of Mode 4 suggests that the stage2 'Routing Outputs' phase can be transformed from a blocking sequential process into a high-speed parallel stream. By integrating this broadcaster, the `stage2_interface.py` script in a downstream operation can begin processing data directly from a kernel pipe as it arrives, enabling pipelined execution across the entire stage2 graph.

Summary

This data suggests minimizing the need for context switching from data between kernel and user space, and elimination of redundant data duplication are the most effective optimization. The `io_uring` framework, when paired with `tee` and `splice`, provides a robust, high-performance pipeline that significantly exceeds the capabilities of standard multi-threaded I/O.

The broadcaster can distribute data to N concurrent operations with near-zero latency added per additional consumer. This "Data Fan-out" capability is essential for real-time co-simulation where solvers, loggers, and analysis tools must synchronize without degrading the solver's performance.

References

- [1] The Open MPI Project. *Open MPI: A High Performance Message Passing Library*. <https://www.open-mpi.org/>
- [2] Introduction to distributed computing with MPI: *MPI Basics* by Tom Nurkkalla. <https://www.youtube.com/>
- [3] Introduction to Parallel Programming using MPI. <https://arcca.github.io/intro-mpi/>
- [4] RookieHPC, learn about High-Performance Computing. <https://rookiehpc.org/>
- [5] Latency vs Jitter in Computer Networks. <https://www.geeksforgeeks.org/>
- [6] Cloudflare internet speed test. <https://speed.cloudflare.com/>
- [7] CAP theorem https://en.wikipedia.org/wiki/CAP_theorem
- [8] Batch vs Stream Processing: *A comprehensive comparison of paradigm*: <https://umamahesh.net/batch-vs-stream-processing-a-comprehensive-comparison-of-paradigms-with-use-cases/>
- [9] Batch vs Stream Processing: *When to Use Each and Why It Matters*. <https://www.datacamp.com/blog/batch-vs-stream-processing>
- [10] Parallel programming in C with MPI and openMP by Michael J. Quinn
- [11] Model-based selection of optimal MPI broadcast algorithms for multi-core clusters. <https://www.sciencedirect.com/science/>
- [12] Liburing: *Library providing helpers for the Linux kernel io_uring support*. <https://github.com/axboe/liburing/tree/master>
- [13] What is io_uring?: Shuveb Hussain, 2020. https://unixism.net/loti/what_is_io_uring.html